

Machine Learning for Trading in the Age of AI Agents

From walk-forward predictions to a cost-aware backtest

Stefan Jansen · *Machine Learning for Trading*, 3rd Edition

RESUME CHECK · 2 MIN

Open the model-training notebook

- Confirm that `data/model_dataset.parquet` exists from Part 1.
- Open `notebooks/03_model_training.ipynb`.
- Keep `notebooks/04_backtest.ipynb` available for the next guided block.

If your Part 1 run did not finish, restart from the executed notebooks supplied in the repository rather than debugging during the session.

Part 2 uses the remaining 98 minutes

Segment	Mode	Time
Train and evaluate the model	Guided <code>03_model_training.ipynb</code>	25 min
Backtest with and without costs	Guided <code>04_backtest.ipynb</code>	30 min
Research-agent forecast	Presenter-led demo	20 min
Production limits and learning paths	Slides	13 min
Q&A	Discussion	10 min

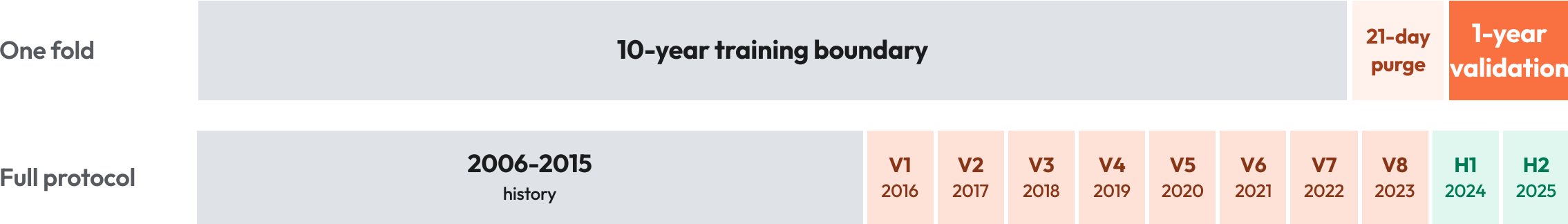
Every reported prediction must be out of sample

- The model estimates 21-day forward ETF returns from the eight features built in Part 1.
- Each test period follows its training period in calendar time.
- Training labels that cross the test boundary are purged.
- Pooled predictions come only from test folds the model did not fit on.

This is a property of the split, not a label attached after training.

Eight validation folds precede a two-year holdout

Each fold moves one year while preserving the same time order



The two holdout years do not participate in model selection or the workshop backtest.

Train across eight validation folds

1. Run through **Walk-forward cross-validation**.
2. Inspect the 10-year `train_size`, `label_horizon=21`, and sealed holdout dates.
3. Let all eight LightGBM fits complete.
4. Compare the first and last train/test windows.

Expected: validation runs from late 2015 through 2023; 2024-2025 remains sealed. Later training windows roll forward instead of expanding from 2008.

Eight folds cover the validation period

8

one-year validation folds

2

sealed holdout years

21

trading-day purge horizon

Each fold contributes 252 validation dates, for 2,016 daily IC observations in total.

Source: executed output, student-repo/notebooks/03_model_training.ipynb

Pg: 7



Information Coefficient evaluates the ranking

- For each date, rank predicted returns across ETFs.
- Rank realized 21-day forward returns across the same ETFs.
- Compute their Spearman correlation: the daily Information Coefficient (IC).
- Average the daily IC series and estimate uncertainty across time.

The metric asks whether higher predicted returns correspond to higher realized returns in the same cross-section.

Compare naive and HAC inference

1. Run **The Information Coefficient**.
2. Inspect the daily IC distribution.
3. Run `compute_ic_hac_stats(..., label_horizon=21)`.
4. Compare `naive_t_stat` with `t_stat`.

Expected: mean IC 0.0152 ; naive t-stat about 3.02 ; HAC-corrected t-stat about 0.94 .

Dependence removes the apparent significance

0.0152

mean daily IC

3.02

naive t-stat

0.94

HAC-corrected t-stat

Consecutive daily ICs share most of the same 21-day return window. The HAC standard error accounts for that dependence; the naive standard error does not.

Source: executed output, student-repo/notebooks/03_model_training.ipynb

Pg: 10

The corrected result changes the decision

- The naive statistic would pass a conventional two-sided 5% threshold.
- The HAC-corrected p-value is about 0.35, which does not distinguish the mean IC from noise.
- Feature importance from the last fold describes that fitted model; it does not establish stable or causal predictors.
- We still backtest the signal to show how a weak model behaves after portfolio construction and costs.

One cross-section becomes ten target weights



Cost fields must be switched on explicitly

```
config = BacktestConfig(  
    initial_cash=100_000.0,  
    commission_type=CommissionType.PER_SHARE,  
    commission_per_share=0.0035,  
    slippage_type=SlippageType.SPREAD,  
    slippage_spread_convention=SpreadConvention.HALF_SPREAD,  
)
```

Setting a rate without the corresponding `*_type` leaves that cost model inactive.

Run the identical strategy twice

1. Run through **The cost model** and confirm `config.validate()` returns an empty list.
2. Inspect the monthly top-10 strategy and trade filters.
3. Run **with and without costs**.
4. Compare Sharpe, return, turnover, total cost, and the trade log.

Expected: 597 cost-aware trades; average turnover about 6.0% per rebalance.

Trading costs reduce return and Sharpe

Metric	Zero cost	With costs
Total return	91.51%	79.14%
Sharpe	0.478	0.440
CAGR	8.46%	7.56%
Maximum drawdown	46.52%	46.57%
Average turnover	5.95%	5.95%
Total commissions + spread	\$0	\$8,597

Source: executed output, student-repo/notebooks/04_backtest.ipynb

Pg: 15



A backtest cannot strengthen the signal

- The HAC-corrected IC t-stat was about 0.94 before portfolio construction.
- The backtest reports the downstream behavior of that weak signal; it does not create independent evidence.
- Monthly rebalancing limits turnover, so the Sharpe reduction is modest relative to the dollar cost.
- The honest conclusion is methodological: inference and execution assumptions determine what survives.

Several live risks remain outside this test

Risk	What this backtest assumes
Slippage	Today's spread schedule remains representative during stressed trading
Capacity	A \$100,000 implementation scales without changing available liquidity
Drift	Scheduled retraining responds quickly enough to regime change
Operations	Data, models, orders, and controls remain synchronized in production

Deployment requires evidence for these assumptions beyond the historical simulation.

PRESENTER-LED DEMONSTRATION · 20 MIN

Research agents produce an inspectable forecast

Chapter 24: specialist estimates, aggregation, adversarial debate, and one calibrated probability.

Independent estimates converge only after disagreement is examined



The final estimate retains the specialist record, disagreement, and debate trace.

Disagreement determines when debate adds value

- Each specialist records a probability and cited reasoning independently.
- Aggregation exposes the size and source of disagreement.
- When estimates diverge, an adversarial phase asks each side to answer the opposing evidence.
- The supervisor produces a final probability after reviewing the full record.

Multiple agents do not guarantee accuracy; they make conflicting assumptions easier to inspect.

Calibration is evaluated after outcomes resolve

- Specialist estimates, aggregation, debate, and the supervisor decision persist in a forecast trace.
- Brier score and log-loss compare probabilities with resolved outcomes.
- The live run uses the presenter's environment because it requires API credentials and external services.
- Attendees can review the design in `docs/research_agent_demo.md` and the book's Chapter 24 companion code.

Choose the next resource by the work you want to do

Goal	Starting point
Reproduce the complete ML4T workflow	Third-edition book + companion repository
Explore a topic before committing	Free Lightning Lessons
Practice the ETF workflow independently	Foundations
Make coding-agent work reliable	Loop Engineering workshop
Build auditable multi-agent forecasts	Multi-Agent Forecasting workshop
Improve and defend a strategy through deployment	Research to Production

The book contains the full research context

- *Machine Learning for Trading*, 3rd Edition develops the workflow across data, modeling, validation, backtesting, and agent systems.
- The companion repository contains the reproducible code, including the complete ETF case study and Chapter 24 agent pipeline.
- The workshop repository retains today's compact four-notebook build and both demonstration briefs.

Resource	Stable destination
Book and companion code	<code>github.com/stefan-jansen/machine-learning-for-trading</code>
ML4T resources and Foundations	<code>ml4trading.io</code>

Continue with free, focused sessions

Lightning Lessons are short, interactive introductions to current topics in strategy research and agent engineering. Use them to inspect a workflow before choosing a longer program.

Examples include:

- Managing the strategy research process.
- Navigating *Machine Learning for Trading*, 3rd Edition.
- Engineering reliable coding-agent loops.
- Diagnosing and improving multi-agent systems.

Current lessons and recordings: maven.com/stefan-jansen

Longer programs separate three objectives

Program	Intended outcome
Foundations	Rebuild the core data-to-backtest workflow at your own pace
Loop Engineering	Design repeatable coding-agent work with explicit planning and verification
Engineering a Multi-Agent Forecasting System	Build, inspect, calibrate, and evaluate a multi-agent forecaster
Machine Learning for Trading: From Research to Production	Diagnose, improve, and defend a strategy through the production boundary

Current catalog: maven.com/stefan-jansen · Full ML4T resources: ml4trading.io

Questions?

Which decision in today's workflow would you verify first on your own strategy?

Thank you

Book, code, Lightning Lessons, workshops, and
courses

`ml4trading.io` · `maven.com/stefan-jansen`